Research Statement

Cross-Layer Co-Design for Next-Generation Datacenter Systems

Inho Choi

1 Overview

I am a systems and networking researcher working at the intersection of academic rigor and industrial practice. Through **cross-layer co-design** across software, hardware, and artificial intelligence (AI), I tackle critical systems challenges and build the next-generation datacenter as a single integrated system, where functionality is placed where it most belongs.

Modern datacenters face a fundamental challenge: serving an ever-expanding spectrum of latency-critical and data-intensive workloads, ranging from microservices and real-time analytics to large-scale AI training and inference. In response, datacenter network fabrics have scaled exponentially, with hyperscalers now entering the era of terabit networking. However, host-side compute has not kept pace. With the end of Moore's Law and Dennard scaling, single-thread CPU performance has plateaued, while the volume and complexity of data each host must process continue to grow. As a result, the host, running applications and systems software, is emerging as the dominant performance bottleneck in modern datacenters.

Meanwhile, two new opportunities are reshaping the datacenter landscape. First, **specialized hardware accelerators** (e.g., SmartNICs, programmable switches, FPGAs, GPUs) are becoming widely available across the datacenter. Each is tailored to a specific class of computation, offering much higher performance and energy efficiency than general-purpose processors. Second, **machine learning** (ML) is increasingly being integrated into core system design. Recent advances have made ML models fast and accurate enough to drive real-time system-level decisions such as parameter tuning and resource allocation.

Approach and directions. Given that the emerging host bottleneck stems from growing compute demands and saturating CPU performance scaling, independently optimizing the software layer faces fundamental limits. This motivates **cross-layer co-design** of modern datacenter systems: carefully distributing functionality across software, hardware, and ML. My research applies this methodology along three connected directions. The first, **co-designing distributed systems with programmable network hardware**, has been the main focus of my doctoral research, where I have introduced programmable network hardware into distributed systems [1, 2, 3, 4]. Building on this foundation, the second, **co-designing operating systems (OSes) with ML**, is my ongoing research, where I am bringing ML into the OS as a first-class system component for real-time adaptive optimization [5]. The third, **co-designing AI infrastructure with hardware accelerators**, is my future direction, where I plan to build systems that coordinate heterogeneous hardware accelerators for AI workloads [6]. Together, these directions form my long-term vision: treating the entire datacenter as a single integrated system where functionality is placed where it most belongs, a design principle that moves next-generation datacenters beyond the limits of single-layer optimization.

Impact and recognition. My research is grounded in both academic rigor and the realities of industry datacenters at scale. Through three research internships at Microsoft Research (MSR) and AMD, I have built systems that target the bottlenecks faced by large-scale production datacenters, with a consistent emphasis on designs that are both technically novel and practically deployable. My work has been recognized by both industry and the research community for not only overcoming the practical deployability and performance limitations of prior state-of-the-art

designs but also inspiring new design paradigms for datacenters. The resulting systems have appeared as first-author publications at top systems and networking venues, including NSDI and SIGCOMM, and have been demonstrated on MSR datacenters. This impact continues through my ongoing close collaboration with Microsoft Research, where I am further advancing my research at the intersection of cutting-edge systems and production datacenters, and Microsoft has expressed interest in integrating my solutions into its datacenter systems.

2 Co-Designing Distributed Systems with Programmable Network Hardware

Traditionally, distributed systems have treated the network as a best-effort fabric that simply forwards and routes packets between hosts. Today, with programmable network hardware, the network can execute custom logic and serve as a primitive for fundamentally new distributed protocols, rather than just a faster substrate for existing ones. My research co-designs distributed systems with network primitives to address three critical datacenter challenges: distributed consensus (Hydra [1]), network load balancing (ConWeave [2]), and server load balancing (Capybara [3, 4]).

2.1 Hydra: Replacing Consensus with Scalable Network Ordering (NSDI '23) [1]

Distributed servers have traditionally relied on consensus algorithms (e.g., Paxos) to maintain a consistent request order, but their expensive per-request coordination has become a key bottleneck in high-performance datacenters. Network ordering systems (e.g., NOPaxos) instead order requests by serializing all requests through a single programmable switch (sequencer) within the network, removing this coordination overhead. However, this single-sequencer design caps system scalability, creates network load imbalance, and makes switch failover prohibitively slow, fundamentally blocking practical adoption.

Hydra is the first work to make network ordering practically deployable at scale. It distributes ordering across *multiple* programmable switches that operate in parallel, with a novel cross-switch ordering scheme that lets receivers merge per-switch sequences without any coordination among the switches themselves. Hydra effectively addresses all three limitations of the single-sequencer design, delivering 11× lower median latency, 47% higher throughput, and 5× faster sequencer failover than prior state-of-the-art network ordering systems.

2.2 ConWeave: Network Load Balancing with In-network Reordering Support for RDMA (SIGCOMM '23) [2]

In modern datacenters, Remote Direct Memory Access (RDMA) powers many performance-critical workloads, from large-scale AI training to distributed storage. Yet RDMA's strict in-order delivery requirement creates a fundamental load balancing problem: balancing load across multiple network paths inevitably causes packets to arrive out of order, and RDMA hardware treats any out-of-order arrival as congestion and immediately slows the sender. This forces datacenters into a difficult trade-off. Coarse-grained load balancing (e.g., per-flow) cannot evenly distribute RDMA traffic, while finer-grained schemes (e.g., per-packet) cause out-of-order arrivals that severely degrade RDMA performance. The trade-off has been a major barrier to fully exploiting RDMA's capacity at scale.

ConWeave is the first work to break this trade-off by performing fine-grained rerouting while restoring packet order *inside the network itself*, before packets reach the destination server. When congestion arises, the sender-side switch reroutes traffic onto a less congested path; the receiver-side switch then uses hardware features of modern programmable switches to temporarily hold and reorder the diverted packets, delivering them in the original order. ConWeave requires

no changes to RDMA hardware or applications, removing a major practical barrier to deployment, while improving 99-percentile flow completion times by up to 66.8% over state-of-the-art load balancing schemes.

2.3 Capybara: Dynamic Server Load Balancing with TCP Migration (APSys '23, SIGCOMM '26) [3, 4]

In datacenters, Layer-4 (L4) load balancers distribute client TCP connections across servers. Because TCP is stateful, each connection must remain pinned to its original server, and L4 load balancers cannot rebalance once connections are established. Due to workload skew across connections, this static pinning fails to fully prevent load imbalance across servers, causing inefficiency in production datacenters.

Capybara is a novel load balancing architecture built on microsecond-scale *live migration* of established TCP connections, enabling dynamic rebalancing of workloads in response to real-time conditions. When the load balancer detects an overloaded server, it can migrate some of its connections to underutilized servers in tens of microseconds. Capybara achieves this through a tight co-design between a programmable network switch that redirects traffic according to migrations and a host TCP stack that can transfer connection states at microsecond speed. Under realistic workloads, Capybara delivers up to 149× lower 99-percentile latency and more than 2× higher throughput compared to state-of-the-art load balancing approaches.

3 Ongoing and Future Research Directions

Building on this foundation, I am extending cross-layer co-design in two further directions: into operating systems and the AI infrastructure layer.

3.1 Co-Designing Operating Systems with Machine Learning (Ongoing)

OS performance tuning is one of the most critical yet difficult problems in datacenter systems (e.g., Google reports that the OS consumes about 20% of all CPU cycles in its datacenters). Thus, major cloud providers invest heavily in dedicated OS optimization teams that tune dozens of parameters across networking, scheduling, and memory subsystems, which interact in non-obvious ways, with optimal settings shifting across workloads and hardware conditions. Modern datacenter workloads have intensified this challenge with their microsecond-scale performance demands and dynamicity, pushing tuning well beyond what fixed rules or human-crafted heuristics can handle. While recent ML advances could address the tuning challenges, the OS itself is not yet designed to host ML as a system component. Conventional OS architectures lack a uniform tuning interface across subsystems, expose no observation points for high-quality training data, and offer no inference path that meets the tight latency budgets of the dataplane, leaving ML as an external optimizer confined to narrow tasks.

Autokernel [5] proposes a new design paradigm for OS architecture that treats ML as a first-class system component. This ML-native architecture provides a uniform tuning interface across subsystems, observation points wired into the dataplane to generate training data, and a hybrid inference path that combines offline lookup tables for well-understood regimes with lightweight ML models for unseen workloads, all within microsecond-scale dataplane processing. Autokernel co-designs the OS with ML, making the adaptive optimization a deployable OS property that future ML policies can build on without re-architecting the integration. While Autokernel establishes how ML should integrate with the OS, the bottlenecks ML must adapt to remain poorly characterized in the community. In a complementary effort, I am empirically characterizing intra-host network bottlenecks, such as the PCIe complex and I/O subsystems. In preliminary evaluation, Autokernel sustains around 100 microseconds of tail latency under dynamic workloads where a statically tuned baseline incurs millisecond-scale queuing delays. Full-system development

is ongoing in close collaboration with Microsoft Research.

Building on these foundations, I plan to advance the principle of ML-native systems along three forward-looking research thrusts:

- **Safe and verifiable learned policies.** Operators need confidence that learned decisions do not violate system-level safety or performance guarantees. I plan to develop a safety framework for ML-native systems where developers declare constraints their ML policies must satisfy (e.g., output ranges, fairness) and fallback actions when constraints are violated (e.g., safe baseline, retraining). These declarations compile into lightweight monitors that enforce the rules at runtime. Instead of each ML-native system reinventing safety checks from scratch, a shared framework lets operators deploy ML policies with the same trust they have in conventional system code.

- **Generative AI for ML-native systems.** Training-data scarcity remains a persistent obstacle for ML-native systems, since production traces are difficult to obtain and rarely shared. Generative AI can synthesize realistic system traces, but faces a fundamental trade-off between expressiveness (capturing diverse patterns) and scalability (learning efficiently at scale). I plan to resolve this trade-off by capturing complex patterns through compact summary variables, and by learning active feedback from self-tuning systems instead of passive traces alone. This approach will produce realistic synthetic traces (packet flows, syscall sequences, scheduling decisions, request patterns) usable for training, evaluation, and what-if analysis in ML systems research.

- **Extending ML-native OS design to other domains.** ML-native design is not specific to traditional host OSes. I plan to apply this principle to GPU host OSes (where memory-allocator, scheduler, and interconnect tuning shape AI workload performance) and embedded/robotics OSes (where adaptive resource management is critical under tight real-time constraints). These extensions will turn ML-native OS into a paradigm that applies to broader domains.

3.2 Co-Designing AI Infrastructure with Hardware Accelerators (Future)

Workloads on modern AI systems, such as LLMs and emerging agentic systems, are fundamentally different from those that shaped previous decades of datacenter design. They run across heterogeneous hardware accelerators (GPU, TPU, FPGA) and memory tiers (HBM, DRAM, CXL), and are increasingly bound by the data movement between them rather than by raw compute. This exposes a growing asymmetry between accelerator compute and inter-device data movement that existing system stacks were not designed to handle.

My exposure to this design space began with RecoNIC [6], an industry collaboration with AMD that gave me direct experience with FPGA-based SmartNIC platforms and with the host–device interaction patterns that dominate AI infrastructure performance. RecoNIC is a hardware–software co-designed platform for RDMA-enabled compute offloading on FPGA-based SmartNICs. The motivation arose from a fundamental inefficiency in conventional FPGA deployments: data arriving from the network is first DMA'd into host memory, then copied over PCIe to the FPGA for processing, and finally copied back to host memory before transmission. These extra hops cause significant performance overhead. RecoNIC addresses this by integrating an RDMA engine and programmable compute-logic modules directly into the SmartNIC's hardware shell, allowing network data to be placed directly into device memory and processed in place by FPGA accelerators.

More broadly, I learned that careful coordination across different system layers is essential for translating advances in individual components like FPGA-based SmartNICs into system-level benefits. Inspired by this, I aim to treat AI infrastructure as a single co-designed system rather

than a collection of independently optimized components. Three connected directions will drive this research: cross-layer dataplanes that orchestrate data movement across accelerators and memory tiers, ML-driven policies that adapt placement and scheduling to workload dynamics, and programmable network hardware that offloads critical-path operations. This vision is a natural synthesis of my prior work, including hardware-software co-design (Hydra, Capybara), ML-native OS (Autokernel), and AI acceleration (RecoNIC), applied to the AI infrastructure that will define the design of next-generation datacenters.

4 Conclusion

Datacenter systems are evolving rapidly. AI-driven workloads, microsecond-scale services, and agentic systems now demand more than incremental tuning or single-layer optimization, while specialized hardware and machine learning expand the design space across layers. My research pursues **cross-layer co-design**: deliberately redistributing system functionality across hardware, software, and machine learning. I carry this methodology across the datacenter, from distributed systems, into the host system itself, and toward the broader AI infrastructure layer. Together, these directions share a single goal: redrawing system boundaries to meet the demands of next-generation datacenters.

References

[1] **Inho Choi**, Ellis Michael, Yunfan Li, Dan R. K. Ports, and Jialin Li. *Hydra: Serialization-Free Network Ordering for Strongly Consistent Distributed Applications.* In Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23), 2023.

[2] Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, **Inho Choi**, Jialin Li, and Mun Choon Chan. *Network Load Balancing with In-network Reordering Support for RDMA.* In Proceedings of the 2023 ACM SIGCOMM Conference (SIGCOMM '23), 2023.

[3] **Inho Choi**, Nimish Wadekar, Raj Joshi, Joshua Fried, Dan R. K. Ports, Irene Zhang, and Jialin Li. *Capybara: µSecond-Scale Live TCP Migration.* In Proceedings of the 14th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '23), 2023.

[4] **Inho Choi**, Nimish Wadekar, Guangda Sun, Raj Joshi, Joshua Fried, Omar S. Navarro Leija, Dan R. K. Ports, Irene Zhang, and Jialin Li. *Capybara: Dynamic Load Balancing with Microsecond-Scale TCP Migration.* In Proceedings of the 2026 ACM SIGCOMM Conference (SIGCOMM '26), 2026.

[5] **Inho Choi**, Anand Bonde, Jing Liu, Joshua Fried, Irene Zhang, and Jialin Li. *ML-native Dataplane Operating Systems.* In Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25), 2025.

[6] Guanwen Zhong, Aditya Kolekar, Burin Amornpaisannon, **Inho Choi**, Haris Javaid, and Mario Baldi. *A Primer on RecoNIC: RDMA-enabled Compute Offloading on SmartNIC.* arXiv:2312.06207, December 2023.